Sameer

+91-9793520310 | md2004sameer01@gmail.com

linkedin.com/in/sameerog | github.com/md2004sameer | leetcode.com/u/sameerog

Summary

Portfolio: me.orbitbot.in

Computer Science undergraduate (graduating 2027) focused on backend engineering with Java and Spring Boot. Builds and deploys REST services and object-oriented systems, backed by 600+ solved LeetCode problems. Seeking entry-level SDE roles and software engineering internships.

Education

Dr. A.P.J. Abdul Kalam Technical University, Lucknow 2023 – 2027
B.Tech. in Computer Science and Engineering

Council for the Indian School Certificate Examinations 2022 – 2023
Senior Secondary, Class XII

Council for the Indian School Certificate Examinations 2020 – 2021
Secondary, Class X

Technical Skills

Languages: Java, SQL

Core CS: Data Structures and Algorithms, OOP, Operating Systems, Computer Networks

Backend: Spring Boot, Spring Data JPA, Hibernate, REST APIs, Multithreading

Databases and Tools: PostgreSQL, MySQL, H2, Redis, Docker, AWS, Git, CI/CD

System Design: Low-Level Design (LLD), High-Level Design (HLD), Distributed Systems

Projects

SquareOne – Workspace

https://github.com/md2004sameer/squareone | https://docs.orbitbot.in/

- Delivers a full-stack document editor with a responsive HTML, CSS, and JavaScript interface and Spring Boot REST APIs for creating, searching, editing, and deleting documents.
- Persists documents with Spring Data JPA using UUID identifiers and creation/update timestamps; debounced autosave batches edits into queued database writes to avoid redundant saves.
- Packages the app as a multi-stage Docker image with Docker Compose, a named volume for durable storage, and a container health check, and serves it live on its own subdomain.
- Tracks revision history per document, supports restoring earlier versions, and runs on PostgreSQL in production.

Chess Engine

https://github.com/md2004sameer/Chess | https://chess.orbitbot.in/

- Implements a Java chess engine with object-oriented design, separating piece-specific move generation, board state, rule-chain move validation, and game-status evaluation.
- Enforces king safety, castling, en passant, promotion, and draw conditions (stalemate, repetition, fifty-move rule, insufficient material) through composable interfaces and polymorphism.
- Verifies correctness with automated tests for legal moves, checkmate, stalemate, and draw scenarios, and documents the architecture and setup in the README.
- Plays against an AI opponent powered by minimax with alpha-beta pruning, and exposes game state through a REST API consumed by the live web interface.

Achievements

- Solved 600+ LeetCode problems and ranked in the platform's top 10%.